
SUPPLEMENTARY MATERIAL FOR STAGED METRIC-GATED GRPO FINE-TUNING PIPELINE FOR VISUAL NUMERIC REASONING

Kunal Ghosh

Purdue University
West Lafayette, Indiana, USA
ghosh178@purdue.edu
kg.aero@gmail.com

1 PURPOSE OF THE SUPPLEMENT

This supplement expands the main paper in five directions:

1. the exact reward and evaluation formulas used by the method,
2. the controller rules and checkpoint-selection logic,
3. the runtime profile used by the reported runs,
4. the final full-evaluation protocol, and
5. the continuation semantics used for same-phase resume versus cross-phase warm start.

All statements below are grounded in the following implementation modules:

- staged_rl/config.py
- staged_rl/rewarding.py
- staged_rl/evaluation.py
- staged_rl/controller.py
- staged_rl/checkpointing.py
- staged_rl/trainer_runtime.py
- rl_gspo_qwen2_5vlm.eval.py

2 GRPO VERSUS THE CONTRIBUTION OF THIS PAPER

Our implementation uses TRL’s GRPOTrainer as its optimization backend. The methodological contribution analyzed in the main paper is the fine-tuning pipeline built around that backend rather than a new GRPO objective:

- staged curriculum construction over filtered MathVista subsets,
- a structured response contract,
- metric-gated reward reweighting after checkpoint evaluation, and
- alias-based checkpoint selection for continuation.

GRPO provides the policy-optimization backbone: the trainer samples multiple completions for a prompt, evaluates them under a reward function, and updates the policy from relative within-group performance rather than from PPO-style clipped ratios. The contribution in this work is the training system built around that optimizer: filtered curricula, metric-gated reward adaptation, checkpoint-aware continuation, and auditable transfer semantics.

3 FULL REWARD SPECIFICATION

Let c denote a completion, g the gold answer, $m \in \{\text{num}, \text{mc}\}$ the answer mode, and L the phase-specific completion budget. Here num denotes the numeric free-form path and mc the multi-choice path. The total per-sample reward is

$$R(c, g, m) = w_f R_{\text{format}} + w_p R_{\text{parseable}} + w_h R_{\text{finished}} + w_c R_{\text{correctness}} + w_b R_{\text{brevity}} + w_t R_{\text{tolerance}}. \quad (1)$$

Tag and parsing predicates. The implementation uses:

$$\text{ReasoningTag}(c) = \mathbf{1}[\text{exactly one reasoning block exists}], \quad (2)$$

$$\text{SolutionTag}(c) = \mathbf{1}[\text{exactly one solution block exists}], \quad (3)$$

$$\text{Malformed}(c, m) = \begin{cases} \mathbf{1}[\text{numeric extraction or parsing fails}], & m = \text{num}, \\ \mathbf{1}[\text{option extraction fails}], & m = \text{mc}, \end{cases} \quad (4)$$

$$\text{Finished}(c, m) = \begin{cases} 1, & \text{ReasoningTag}(c) = 1 \wedge \text{SolutionTag}(c) = 1 \\ & \wedge \text{Malformed}(c, m) = 0, \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

Reward components. The six reward components are:

$$R_{\text{format}}(c, m) = \text{ReasoningTag}(c) + \text{SolutionTag}(c) - 0.5 \text{Malformed}(c, m), \quad (6)$$

$$R_{\text{parseable}}(c, m) = \begin{cases} \mathbf{1}[\text{numeric parsing succeeds}], & m = \text{num}, \\ \mathbf{1}[\text{Malformed}(c, m) = 0], & m = \text{mc}, \end{cases} \quad (7)$$

$$R_{\text{finished}}(c, m) = \begin{cases} 1, & \text{Finished}(c, m) = 1 \wedge \text{tok}(c) < L, \\ -1, & \text{tok}(c) \geq L \wedge \text{Finished}(c, m) = 0, \\ 0, & \text{otherwise,} \end{cases} \quad (8)$$

$$R_{\text{correctness}}(c, g, m) = \mathbf{1}[\text{exact match}], \quad (9)$$

$$R_{\text{brevity}}(c) = \begin{cases} -1, & \text{tok}(c) \geq L, \\ 1, & \text{tok}(c)/L \leq 0.55, \\ 0.5, & 0.55 < \text{tok}(c)/L \leq 0.75, \\ 0, & 0.75 < \text{tok}(c)/L \leq 0.90, \\ -0.5, & 0.90 < \text{tok}(c)/L < 1, \end{cases} \quad (10)$$

$$R_{\text{tolerance}}(c, g, m) = \begin{cases} 1, & \text{tolerance is enabled, exact match is false,} \\ & \text{and the tolerance check holds,} \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

Tolerance reward is active in Phase C. Exact match suppresses tolerance reward, so the tolerance component acts strictly as a denser near-miss signal rather than an additional bonus for already-correct answers.

Phase-initialized weights. The configured initial weights for the reported training phases are manual curriculum defaults rather than values learned by a meta-optimizer:

Phase	Format	Parseable	Finished	Correctness	Brevity	Tolerance
A	1.0	1.0	1.5	2.0	0.25	0.0
B	0.75	0.75	1.0	4.0	0.20	0.0
C	0.50	0.50	0.75	5.0	0.20	1.0

Table 1: Controller rules implemented in `staged_rl/controller.py`.

Rule	Trigger	Action
Parseable guard	parseable answer rate < 0.85	Raise parseable reward to at least 0.75
Format guard	solution tag < 0.90 or reasoning tag < 0.88 or malformed rate > 0.10	Raise format reward to at least 0.75
Finish guard	truncation rate > 0.15 or average completion tokens exceed $0.85L$	Add 0.25 to finished reward
Stable structure test	parseable ≥ 0.92 , solution ≥ 0.95 , reasoning ≥ 0.93 , malformed ≤ 0.05 , truncation ≤ 0.08	Enable plateau-based correctness escalation once enough history exists
Correctness escalation	Stable structure holds over a two-checkpoint window and exact-match gain is below 0.02	Add 0.50 to correctness reward

4 EVALUATION METRICS AND CHECKPOINT SCORES

For each prompt i , the evaluator samples one or more completions $c_{i,j}$. The final reported evaluation uses one completion per prompt. The evaluator computes:

$$\text{EM} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{first sampled completion matches exactly}], \quad (12)$$

$$\text{TolAcc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{first sampled completion is correct under tolerance}], \quad (13)$$

$$\text{ParseRate} = \frac{1}{M} \sum_{i,j} \mathbf{1}[\text{sample } (i, j) \text{ is parseable}], \quad (14)$$

$$\text{MalRate} = \frac{1}{M} \sum_{i,j} \mathbf{1}[\text{sample } (i, j) \text{ is malformed}], \quad (15)$$

$$\text{TruncRate} = \frac{1}{M} \sum_{i,j} \mathbf{1}[\text{sample } (i, j) \text{ is truncated}]. \quad (16)$$

It also computes tag-compliance rates, completion-token averages, repetition rate, sampled-answer diversity, and mean logged reward components.

The checkpoint registry then defines three scores:

$$S_{\text{struct}} = 0.35 \text{ ParseRate} + 0.20 \text{ SolutionTag} + 0.20 \text{ ReasoningTag} \\ - 0.15 \text{ MalRate} - 0.10 \text{ TruncRate}, \quad (17)$$

$$S_{\text{corr}} = 0.70 \text{ EM} + 0.20 \text{ TolAcc} + 0.10 \text{ ParseRate}, \quad (18)$$

$$S_{\text{comp}} = 0.45 \text{ EM} + 0.15 \text{ TolAcc} + 0.15 \text{ ParseRate} + 0.10 \text{ SolutionTag} \\ + 0.05 \text{ ReasoningTag} - 0.05 \text{ MalRate} - 0.05 \text{ TruncRate}. \quad (19)$$

These scores back the aliases `best_structure`, `best_correctness`, and `best_composite`. The latest alias is purely chronological.

5 CONTROLLER RULES AND THRESHOLDS

The controller rules are intentionally concise and inspectable. The main paper does not rely on a fixed count of controller firings because the final primary comparison is based on selected checkpoints reevaluated on the full numeric evaluation subset.

6 RUNTIME PROFILE AND FINAL EVALUATION PROTOCOL

The reported training and final evaluation share the same memory-constrained adapter and generation budget:

- max sequence length 1280,
- image size 336,
- LoRA rank 8 with LoRA alpha 8,
- max prompt length 320,
- max completion length 64,
- two sampled generations per prompt during GRPO fine-tuning,
- one sampled completion per prompt during final evaluation, and
- full final evaluation with no per-subset example cap.

Runtime configuration. The same runtime profile can be read directly in the style of the code paths that consume these settings:

```
# Model load / LoRA adapter profile
FastVisionModel.from_pretrained(
    model_name="unsloth/Qwen2.5-VL-7B-Instruct",
    max_seq_length=1280,
    load_in_4bit=True,
    fast_inference=True,
    gpu_memory_utilization=0.65,
    max_lora_rank=8,
    compilation_config={"level": 3, "cudagraph_mode": "PIECEWISE"},
)

FastVisionModel.get_peft_model(
    model,
    r=8,
    lora_alpha=8,
    finetune_vision_layers=False,
    finetune_language_layers=True,
    finetune_attention_modules=True,
    finetune_mlp_modules=True,
)
```

```
# Trainer / final evaluation profile
GRPOConfig(
    gradient_accumulation_steps=4,
    num_generations=2,
    max_prompt_length=320,
    max_completion_length=64,
    importance_sampling_level="sequence",
    loss_type="dr_grpo",
)

SamplingParams(
    temperature=1.0,
    top_k=50,
    max_tokens=64,
    n=1,
)

EvalConfig(
    num_samples_per_prompt=1,
    max_eval_examples_per_subset=None,
)
```

Final evaluation target set. The final comparison evaluates four targets:

Table 2: Final primary evaluation on the full `eval_overall_numeric` subset.

Phase	Checkpoint	Exact	Tol	Parse	AvgR	Struct	Corr	Comp
Baseline	<code>baseline_rank8</code>	0.1798	0.1820	0.2544	-0.0880	0.0285	0.1877	0.1163
Phase A	<code>checkpoint-237</code>	0.4803	0.4868	0.9890	6.3857	0.7421	0.5325	0.5859
Phase B	<code>checkpoint-180</code>	0.4978	0.5066	0.9781	5.8754	0.7371	0.5476	0.5946
Phase C	<code>checkpoint-240</code>	0.4737	0.4846	0.9890	6.0361	0.7436	0.5274	0.5833

Table 3: Full stage-wise diagnostic results on MathVista `testmini`.

Phase	Eval subset	Exact	Tol	Parse	Malf	Trunc	AvgR
Baseline	Stage 1	0.1641	0.1641	0.2872	0.7128	0.6923	0.1981
Baseline	Stage 2	0.1220	0.1220	0.1672	0.8328	0.8293	-0.7321
Baseline	Stage 3	0.0924	0.0924	0.1681	0.8319	0.8151	-0.8057
Phase A	Stage 1	0.4615	0.4615	0.9897	0.0103	0.0051	6.3417
Phase A	Stage 2	0.4321	0.4460	0.9895	0.0105	0.0000	6.2666
Phase A	Stage 3	0.3950	0.4034	0.9832	0.0168	0.0084	6.1261
Phase B	Stage 1	0.4718	0.4718	0.9846	0.0154	0.0000	5.7786
Phase B	Stage 2	0.4774	0.4983	0.9721	0.0279	0.0035	5.7652
Phase B	Stage 3	0.3866	0.4118	0.9664	0.0336	0.0000	5.3517
Phase C	Stage 1	0.4564	0.4564	0.9692	0.0308	0.0000	5.8790
Phase C	Stage 2	0.4321	0.4530	0.9861	0.0139	0.0000	5.8103
Phase C	Stage 3	0.4202	0.4286	0.9832	0.0168	0.0000	5.7294

Label	Checkpoint
Baseline	<code>baseline_rank8</code>
Phase A	<code>phase_a/checkpoint-237</code>
Phase B	<code>phase_b/checkpoint-180</code>
Phase C	<code>phase_c/checkpoint-240</code>

The primary subset is `eval_overall_numeric` from MathVista `testmini`. Its size is 456 examples. The diagnostic stage subsets contain 195 Stage 1 examples, 287 Stage 2 examples, and 119 Stage 3 examples.

7 FULL EVALUATION RESULTS

8 TRANSFER AND CONTINUATION SEMANTICS

The code distinguishes three mechanisms that can be conflated in informal descriptions:

1. **Artifact transfer.** A checkpoint directory is made available to a later run as an input artifact.
2. **Warm start.** A fresh base model is loaded, a fresh LoRA adapter is attached, and adapter weights are copied from the selected checkpoint into that new adapter.
3. **Same-phase resume.** Full trainer state is restored from a checkpoint, including optimizer state, scheduler state, trainer progress, and controller state.

This distinction is important for interpreting the reported checkpoints. Cross-phase continuation uses selected adapter weights as a warm start. Same-phase restarts can use exact trainer resume if the selected checkpoint belongs to the current phase and the full trainer state is available.

9 ADDITIONAL SCOPE NOTES

The implementation contains preliminary support for a multi-choice branch and an optional robustness stage, but neither path provides the main evidence used in the paper. Similarly, the runner logs when multiple GPUs are visible but does not automatically switch to DDP or tensor parallelism. The primary claims are therefore:

- about staged GRPO fine-tuning pipeline dynamics for numeric visual reasoning,

-
- about metric-gated control under non-monotonic checkpoint trajectories, and
 - about making a memory-constrained GRPO fine-tuning pipeline more auditable and reproducible.